

Spesifikasi Tugas Besar Inteligensi Artifisial

Local Search untuk Persoalan Penjadwalan Mata Kuliah
Hill-Climbing · Simulated Annealing · Genetic Algorithm

Task #2 Seleksi Laboratorium Inteligensi Buatan

Tengku Naufal Saqib 13524012

Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung

Dokumen ini terdiri atas	Halaman
Bagian I Spesifikasi Tugas Besar: Local Search	1 - 12
Bagian II Write-up DTL, Logistic Regression, dan SVM	13 - 17

Kode sumber, notebook eksperimen, dan seluruh berkas hasil tersedia pada repository github.com/tengkuNaufal/Task2_AI_13524012.

Bagian I Spesifikasi Local Search

1. Tujuan dan Deskripsi Permasalahan

Setiap awal semester, tim akademik harus menyusun jadwal perkuliahan: menempatkan setiap kelas mata kuliah pada sebuah **slot waktu** dan sebuah **ruangan**. Penjadwalan ini sulit karena sumber daya terbatas, ruangan tidak boleh dipakai dua kelas sekaligus, seorang dosen tidak dapat mengajar di dua tempat pada jam yang sama, dan mahasiswa satu angkatan tidak boleh punya dua kuliah bertabrakan. Di luar aturan keras itu masih ada banyak preferensi: dosen enggan mengajar pada slot tertentu, mahasiswa tidak ingin jadwalnya berlubang, dan ruang besar sebaiknya tidak dipakai kelas kecil.

Tujuan tugas besar ini adalah menyusun jadwal yang memenuhi seluruh aturan keras dan meminimalkan pelanggaran preferensi, dengan pendekatan **local search**.

Persoalan ini memenuhi syarat local search karena **yang dicari adalah konfigurasinya, bukan jalan menuju konfigurasi tersebut**. Tidak ada makna "langkah pertama" pada sebuah jadwal: yang dinilai hanyalah jadwal akhirnya. Karena itu masalah ini dapat dirumuskan sebagai *complete-state formulation*, setiap state sudah berupa jadwal utuh, dan pencarian bergerak dari satu jadwal utuh ke jadwal utuh lain yang bertetangga.

1.1 Instansi yang digunakan

Instansi bawaan meniru satu semester di STEI ITB dan bersifat deterministik agar hasil eksperimen dapat direproduksi.

Komponen	Jumlah	Keterangan
Kelas mata kuliah (n)	36	24 kuliah teori + 12 praktikum/lab
Ruangan ($ R $)	11	8 ruang kuliah (kapasitas 40–120) + 3 laboratorium (40–60)
Slot waktu ($ T $)	25	5 hari (Senin–Jumat) $\times$ 5 sesi @2 jam (07.00–17.00)
Dosen	10	tiap dosen punya 3 slot yang dinyatakan tidak tersedia
Kelompok mahasiswa	8	IF/STI/EL angkatan 2023–2024, per kelas

Ukuran ruang pencarian adalah $(|T| \times |R|)^n = 275^{36} \approx 1,2 \times 10^{87}$ konfigurasi. Enumerasi menyeluruh jelas mustahil, sehingga pencarian lokal menjadi pilihan yang wajar.

2. Representasi State

State direpresentasikan sebagai vektor berukuran tetap sepanjang jumlah mata kuliah. Gen ke- i menyimpan pasangan slot waktu dan ruangan untuk mata kuliah ke- i :

```
X = [ (t_0, r_0), (t_1, r_1), ..., (t_35, r_35) ]

t_i in {0, 1, ..., 24}   slot waktu   (t = 5*hari + sesi)
r_i in {0, 1, ..., 10}   indeks ruangan

Contoh gen: (7, 2)  ->  hari = 7 // 5 = 1 (Selasa), sesi = 7 % 5 = 2 (11.00-13.00),
                      ruang = indeks 2 (7603)
```

Tiga sifat penting dari representasi ini:

- **Lengkap.** Setiap state adalah jadwal utuh, seluruh 36 mata kuliah selalu memiliki penugasan. Tidak ada state "setengah jadi", sehingga fungsi objektif dapat dihitung kapan saja.
- **Panjang tetap.** Karena panjangnya tidak berubah, operator crossover dan mutasi pada Genetic Algorithm selalu menghasilkan jadwal yang valid secara struktur, tanpa perlu mekanisme perbaikan (*repair*).
- **Tanpa constraint bawaan.** Seluruh aturan ditegakkan lewat fungsi objektif, bukan lewat pembatasan domain. Ini membuat ruang pencarian terhubung penuh: setiap state dapat dicapai

dari state mana pun, sehingga pencarian tidak pernah terkurung di daerah yang terisolasi.

3. Aturan dan Batasan Permasalahan

Batasan dibagi menjadi **hard constraint** (wajib; jadwal yang melanggarnya tidak dapat dijalankan) dan **soft constraint** (preferensi; pelanggarannya menurunkan kualitas tetapi jadwal tetap dapat dipakai).

3.1 Hard constraint

Kode	Aturan	Bobot
H1	Bentrok ruangan , tidak boleh ada dua kelas pada pasangan (slot, ruang) yang sama.	100
H2	Bentrok dosen , seorang dosen tidak boleh mengampu dua kelas pada slot yang sama.	100
H3	Bentrok mahasiswa , satu kelompok mahasiswa tidak boleh memiliki dua kuliah pada slot yang sama.	100
H4	Kapasitas ruangan , jumlah peserta tidak boleh melebihi kapasitas ruangan.	60
H5	Jenis ruangan , kelas praktikum wajib ditempatkan di laboratorium.	80

3.2 Soft constraint

Kode	Preferensi	Bobot
S1	Dosen tidak dijadwalkan pada slot yang ia nyatakan tidak tersedia.	10
S2	Hindari sesi terakhir hari itu (15.00–17.00).	3
S3	Dosen mengajar paling banyak 2 kelas per hari (penalti per kelebihan).	5
S4	Satu kelompok mahasiswa kuliah paling banyak 3 sesi per hari.	4
S5	Jadwal harian kelompok mahasiswa tidak berlubang (penalti per jam kosong di antara kuliah pertama dan terakhir).	2
S6	Jadwal harian dosen tidak berlubang (penalti per jam kosong).	2
S7	Kursi menganggur diminimalkan (penalti per kelipatan 10 kursi kosong).	1
S8	Ruang laboratorium tidak dipakai untuk kuliah teori.	4

Sebuah jadwal disebut **feasible** bila seluruh hard constraint terpenuhi, yaitu H1–H5 semuanya bernilai nol.

4. Formulasi Initial State

Initial state dibangkitkan **sepenuhnya secara random**. Untuk setiap mata kuliah i , slot waktu dan ruangan diundi seragam dan saling bebas:

```
def random_state(rng):  
    return [ (rng.randrange(25), rng.randrange(11)) for _ in range(36) ]
```

Tidak ada perbaikan (*repair*) atau heuristik penempatan awal apa pun. Pilihan ini disengaja karena tiga alasan. **Pertama**, state awal yang buruk membuat kemampuan tiap algoritma untuk memperbaiki keadaan terlihat jelas. **Kedua**, semua algoritma dapat diberi state awal yang identik sehingga perbandingannya adil. **Ketiga**, pembangkitan acak memungkinkan eksperimen multi-seed untuk mengukur kestabilan algoritma, bukan sekadar keberuntungan satu kali jalan.

Sebagai gambaran, state awal dengan *seed* 42 memiliki $f(X) = 2015$ dengan 24 pelanggaran hard constraint, sangat jauh dari jadwal yang dapat dipakai.

5. Successor Function dan Langkah Neighbor

Successor function membangkitkan seluruh tetangga sebuah state. Tiga jenis *move* diperbolehkan pada setiap iterasi:

Move	Definisi	Jumlah tetangga	Peran
change-time (i, t)	Pindahkan mata kuliah <i>i</i> ke slot waktu <i>t</i> ; ruangan tetap.	$n \times (T - 1)$ $= 36 \times 24 = \mathbf{864}$	Menyelesaikan bentrok waktu (H2, H3) dan pelanggaran preferensi jam (S1–S6).
change-room (i, r)	Pindahkan mata kuliah <i>i</i> ke ruangan <i>r</i> ; slot waktu tetap.	$n \times (R - 1)$ $= 36 \times 10 = \mathbf{360}$	Menyelesaikan bentrok ruang (H1), kapasitas (H4), dan jenis ruang (H5, S7, S8).
swap (i, j)	Tukar pasangan (slot, ruang) mata kuliah <i>i</i> dan <i>j</i> .	$n(n - 1)/2$ $= \mathbf{630}$	Menukar dua kelas sekaligus tanpa merusak okupansi total; berguna saat jadwal sudah padat dan tidak ada slot kosong yang tersisa.

Total ukuran neighborhood adalah **1.854 state** per iterasi. Ketiga jenis move saling melengkapi: dua move pertama menggeser satu kelas saja (perubahan halus), sedangkan *swap* memungkinkan pertukaran yang tidak dapat dicapai oleh satu gerakan tunggal ketika seluruh slot alternatif sudah terisi, tanpa *swap*, pencarian mudah tersangkut pada jadwal padat.

Algoritma *steepest-ascent* memeriksa seluruh 1.854 tetangga tiap iterasi, sedangkan Stochastic Hill-Climbing, Simulated Annealing, dan operator mutasi Genetic Algorithm hanya mengundi satu tetangga acak (peluang 45% change-time, 35% change-room, 20% swap).

6. Objective Function

Dipakai satu **fungsi biaya tunggal yang diminimalkan**, berupa jumlah berbobot seluruh pelanggaran:

```

f(X) = SUM_{h in H1..H5} w_h * v_h(X)  +  SUM_{s in S1..S8} w_s * v_s(X)

v_c(X) = cacah pelanggaran batasan c pada state X
w_c    = bobot batasan c (lihat tabel Bagian 3)

f(X) >= 0;  f(X) kecil =>  jadwal lebih baik
X feasible <=>  v_H1 = ... = v_H5 = 0

```

6.1 Alasan pemilihan

- **Mengapa fungsi biaya, bukan fungsi nilai?** Pelanggaran secara alami dihitung sebagai bilangan bulat tak negatif, dan jadwal ideal berpadanan dengan nilai nol. Merumuskannya sebagai biaya membuat batas bawah teoretisnya jelas dan interpretasinya langsung: "berapa besar total kerugian jadwal ini".
- **Mengapa satu skalar, bukan multi-objektif?** Hill-Climbing dan Simulated Annealing membutuhkan pengurutan total antar state. Skalarisasi berbobot memberikan pengurutan itu sekaligus tetap dapat didekomposisi, implementasi tetap melaporkan cacah tiap komponen untuk keperluan audit.
- **Mengapa bobot hard constraint jauh lebih besar?** Bobot terkecil untuk hard constraint adalah 60, sedangkan penalti lunak terbesar hanya 10. Dengan begitu menukar satu pelanggaran keras demi perbaikan lunak sebesar apa pun yang masuk akal tidak akan pernah menguntungkan, pencarian dipaksa mengejar kelayakan terlebih dahulu, baru kenyamanan.
- **Mengapa H1-H3 diberi bobot lebih tinggi daripada H4-H5?** Bentrok jadwal membuat kuliah benar-benar tidak dapat berlangsung, sedangkan kelebihan kapasitas atau salah jenis ruang masih dapat diakali di lapangan (menambah kursi, meminjam ruang lain).
- **Mengapa bertahap, bukan biner?** Setiap komponen mencacah pelanggaran, bukan sekadar menandai ada/tidak ada. Ini menghasilkan permukaan pencarian yang bergradasi sehingga algoritma memperoleh sinyal perbaikan pada hampir setiap langkah, fungsi biner akan menciptakan plateau raksasa yang membuat Hill-Climbing lumpuh sejak iterasi pertama.

Konsekuensi praktis dari rancangan ini: nilai $f(\mathbf{X}) = 0$ tidak tercapai pada instansi bawaan karena beberapa preferensi saling bertentangan (misalnya S5 menuntut jadwal mahasiswa rapat, sementara S3 membatasi beban harian dosen). Eksperimen menunjukkan nilai terbaik yang berulang kali dicapai adalah $f(\mathbf{X}) = 16$, seluruhnya berupa penalti kursi menganggur (S7), yang memang tidak dapat dihilangkan karena kapasitas ruangan bersifat diskret.

7. Algoritma yang Wajib Diimplementasikan

7.1 Hill-Climbing (keempat varian)

Varian	Aturan pemilihan langkah	Kriteria berhenti
Basic (Steepest-Ascent)	Evaluasi seluruh 1.854 tetangga, pindah ke yang f -nya paling kecil.	Tidak ada tetangga dengan f lebih kecil (local optimum).
Sideways Move	Sama seperti Steepest-Ascent, tetapi bila tidak ada perbaikan, pindah ke tetangga yang f -nya sama (dipilih acak).	Tidak ada perbaikan dan kuota langkah mendatar berturut-turut (40) habis.
Stochastic	Undi satu tetangga acak; terima hanya bila f -nya lebih kecil.	Batas iterasi tercapai (30.000).
Random Restart	Jalankan Steepest-Ascent dari beberapa state awal acak, ambil hasil terbaik.	Seluruh restart (5×) selesai.

7.2 Simulated Annealing

Tiap iterasi mengundi satu tetangga acak. Bila lebih baik, langsung diterima. Bila lebih buruk sebesar ΔE , tetangga tetap diterima dengan peluang $P = e^{-\Delta E/T}$. Suhu diturunkan secara geometrik $T \leftarrow \alpha T$ dengan $T_0 = 60$ dan $\alpha = 0,9997$, dan pencarian berhenti saat $T < 10^{-3}$ atau batas iterasi tercapai.

Pada suhu tinggi P mendekati 1 sehingga SA berkelana bebas (eksplorasi); pada suhu rendah P mendekati 0 sehingga perilakunya menyatu dengan Hill-Climbing biasa (eksploitasi). Implementasi mencatat setiap nilai P untuk keperluan analisis, serta menghitung berapa kali SA menolak langkah memburuk saat suhu sudah di bawah 1, proksi seberapa sering ia "terjebak" di akhir pendinginan.

7.3 Genetic Algorithm

Komponen	Rancangan
Kromosom	Persis representasi state: vektor 36 gen, gen ke- i = (slot, ruang) mata kuliah ke- i . Panjang tetap $\Rightarrow$ keturunan selalu valid secara struktur.
Populasi	80 individu, seluruhnya dibangkitkan acak (kecuali satu individu yang disetel ke state awal bersama agar sebanding dengan algoritma lain).
Fitness	$-f(\mathbf{X})$. Semakin besar semakin baik.
Selection	<i>Tournament selection</i> ukuran 3, ambil 3 individu acak, menangkan yang f -nya terkecil. Tekanan seleksi moderat sehingga keragaman terjaga.
Crossover	<i>Uniform crossover</i> (peluang 0,9): tiap gen ditukar antar kedua induk dengan peluang 0,5. Tersedia pula <i>one-point crossover</i> sebagai pembanding.
Mutation	Peluang 0,2 per keturunan: satu gen acak diganti pasangan (slot, ruang) acak, setara satu langkah <i>change-both</i> .
Elitism	2 individu terbaik disalin utuh ke generasi berikutnya sehingga solusi terbaik tidak pernah hilang.
Berhenti	Setelah 400 generasi.

8. Daftar Fungsi dan Command Utama

8.1 Command line

<code>python -m local_search.main --algo all --seed 42 --plot --animate</code>
--

```
--algo {hc-steepest | hc-sideways | hc-stochastic | hc-random-restart | sa | ga | all}
--seed N          seed RNG (state awal & keputusan acak) -- reproduisibel
--instance FILE.json memuat instansi lain; default instansi bawaan
--out DIR          direktori keluaran (default results/local_search)
--max-iter N       batas iterasi varian Hill-Climbing (default 5000)
--max-sideways N   kuota langkah mendatar berturut-turut (default 40)
--stochastic-iter N batas iterasi Stochastic HC (default 30000)
--restarts N       jumlah restart Random-Restart HC (default 5)
--sa-t0 F --sa-alpha F --sa-tmin F --sa-iter N      parameter Simulated Annealing
--pop N --gen N --mutation F --crossover-rate F --elitism N --crossover {uniform,one_point}
--plot            simpan grafik objective, diagnostik SA/GA, dan heatmap
--animate         simpan animasi GIF perubahan state antar iterasi
--quiet           tekan cetak jadwal lengkap
--save-instance FILE simpan instansi bawaan ke JSON lalu keluar
```

8.2 Fungsi utama

Modul / fungsi	Penjelasan
problem.py	
TimetableProblem	Instansi masalah: daftar mata kuliah, ruangan, dosen, kelompok, bobot, dan ketidaktersediaan dosen. Menyediakan pemuatan/penyimpanan JSON.
random_state(rng)	Membangkitkan initial state acak (Bagian 4).
counts(state)	Mencacah pelanggaran mentah tiap komponen H1-H5 dan S1-S8.
objective(state)	Mengembalikan nilai $f(X)$, versi ringan yang dipanggil jutaan kali ($\approx 39 \mu s$).
evaluate(state)	Mengembalikan objek Cost berisi total, rincian per komponen, status feasible, dan laporan teks.
neighbors(state)	Successor function: generator seluruh 1.854 tetangga beserta move-nya.
random_neighbor(state, rng)	Satu tetangga acak, dipakai Stochastic HC, SA, dan mutasi GA.
algorithms.py	
hill_climbing_steepest	Basic / Steepest-Ascent Hill-Climbing.
hill_climbing_sideways	Hill-Climbing dengan langkah mendatar berkuota.
hill_climbing_stochastic	Stochastic Hill-Climbing.
hill_climbing_random_restart	Random-Restart Hill-Climbing.
simulated_annealing	Simulated Annealing; mengembalikan pula riwayat suhu dan seluruh peluang penerimaan.
genetic_algorithm	Genetic Algorithm; mengembalikan riwayat f terbaik, rata-rata, dan terburuk per generasi.
SearchResult	Rangkuman satu eksekusi: state awal/akhir, biaya, riwayat, jumlah iterasi dan evaluasi, durasi.
visualize.py	
render_timetable	Cetak jadwal sebagai matriks hari $\times$ jam.
render_assignment	Daftar penugasan per mata kuliah, terurut menurut waktu.
render_violations	Rincian pelanggaran hard constraint yang tersisa (audit hasil).
plot_history	Grafik $f(X)$ terhadap iterasi untuk beberapa algoritma sekaligus.
plot_sa_diagnostics	Tiga panel SA: $f(X)$, suhu, dan peluang penerimaan $e^{-\Delta E/T}$.
plot_ga_diagnostics	Kurva f terbaik / rata-rata / terburuk populasi tiap generasi.
animate_search	Animasi GIF perubahan okupansi ruangan antar iterasi.

9. Contoh Input dan Output

9.1 Input

```
$ python -m local_search.main --algo hc-steepest --seed 42 --quiet
```

9.2 Output

```

=====
PoC LOCAL SEARCH - PENJADWALAN MATA KULIAH
=====
Instansi      : STEI-ITB (36 kelas / 11 ruang / 25 slot)
Mata kuliah (n) : 36
Ruangan (|R|)  : 11
Slot waktu (|T|) : 25 (5 hari x 5 jam)
Ukuran ruang state : (25 x 11)^36
Ukuran neighborhood: 1854 state per iterasi
Seed          : 42

-----
STATE AWAL (dibangkitkan secara random)
-----
Objective f(X) = 2015
Hard constraints (bobot besar):
  room_clash      1 x 100 = 100
  lecturer_clash  0 x 100 = 0
  group_clash     4 x 100 = 400
  capacity        13 x 60 = 780
  room_type       6 x 80  = 480
Soft constraints:
  lecturer_unavailable 10 x 10 = 100
  late_slot            6 x 3  = 18
  lecturer_overload    1 x 5  = 5
  group_overload       3 x 4  = 12
  group_gap            7 x 2  = 14
  lecturer_gap         8 x 2  = 16
  room_waste           66 x 1  = 66
  lab_misuse           6 x 4  = 24
-> total pelanggaran keras = 24 (TIDAK FEASIBLE), penalti lunak = 255

-----
MENJALANKAN: Basic / Steepest-Ascent Hill-Climbing
-----
[Hill-Climbing (Steepest-Ascent)]
objective awal   : 2015
objective akhir  : 18 (FEASIBLE)
perbaikan       : 1997
iterasi         : 41
evaluasi f(X)   : 77866
durasi          : 3.483 s
berhenti_karena : local optimum

```

Bila dijalankan tanpa --quiet, program juga mencetak jadwal awal dan jadwal akhir dalam bentuk matriks hari x jam berikut daftar penugasan lengkap. Cuplikan jadwal akhir terbaik:

Jam	Senin	Selasa	Rabu	Kamis	Jumat
07.00-09.00	IF3170@7602 -	- -	STI2210@7612 -	- -	IF2123@7602 IF3170-P@Lab-Dasar
09.00-11.00	IF2130@7602 IF2240@7603 - -	IF2110-P@Lab-Dasar IF2220@7603 - -	IF3110@Lab-Dasar STI2220@7612 - -	IF2110@7602 EL2140-P@Lab-IRK - -	IF2120@7602 IF2211-P@Lab-Dasar IF3140@7611 STI2230@7612
11.00-13.00	IF2124@7602 IF3210@7611 IF3230@7610 -	IF2130-P@Lab-Dasar IF2211@7603 STI2110@9010 -	IF2210-P@Lab-Dasar IF3260@Lab-IRK - -	IF2230@7602 EL3140@Lab-Grafika EL3240@7612 KU2071@7502	IF3130@7610 STI2120@7612 - -
13.00-15.00	- -	STI2130@9010 EL2140@9009	IF2210@7603 EL2240@9010	IF2240-P@Lab-Dasar MA2071@9009	- -
15.00-17.00	-	-	-	-	-

Tidak ada pelanggaran hard constraint. Jadwal FEASIBLE.

Perhatikan bahwa baris 15.00–17.00 kosong seluruhnya, algoritma berhasil memenuhi preferensi S2 tanpa diberi tahu secara eksplisit bahwa slot itu harus dikosongkan.

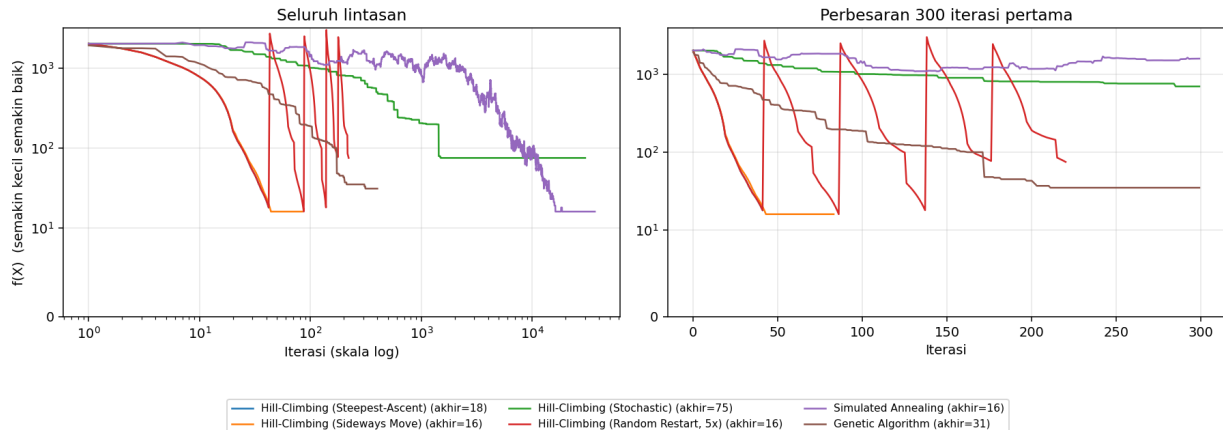
10. Hasil Proof of Concept

Keenam algoritma dijalankan dari **state awal yang identik** (seed 42, f = 2015).

Algoritma	f akhir	Pelanggaran keras	Iterasi	Evaluasi f(X)	Detik
Hill-Climbing (Steepest-Ascent)	18	0	41	77.866	3,48

Algoritma	f akhir	Pelanggaran keras	Iterasi	Evaluasi f(X)	Detik
Hill-Climbing (Sideways Move)	16	0	83	155.734	7,14
Hill-Climbing (Stochastic)	75	1	30.000	30.001	1,40
Hill-Climbing (Random Restart, 5x)	16	0	216	409.728	18,35
Simulated Annealing	16	0	36.669	36.670	1,71
Genetic Algorithm	31	0	400	32.080	1,55

Nilai objective function terhadap iterasi



Gambar 1. Nilai objective function terhadap iterasi. Panel kiri memakai sumbu-x logaritmik agar keenam lintasan yang panjangnya berbeda ekstrem dapat dibandingkan; panel kanan memperbesar 300 iterasi pertama. Gerigi tajam pada kurva Random Restart adalah momen pencarian dimulai ulang dari state acak baru.

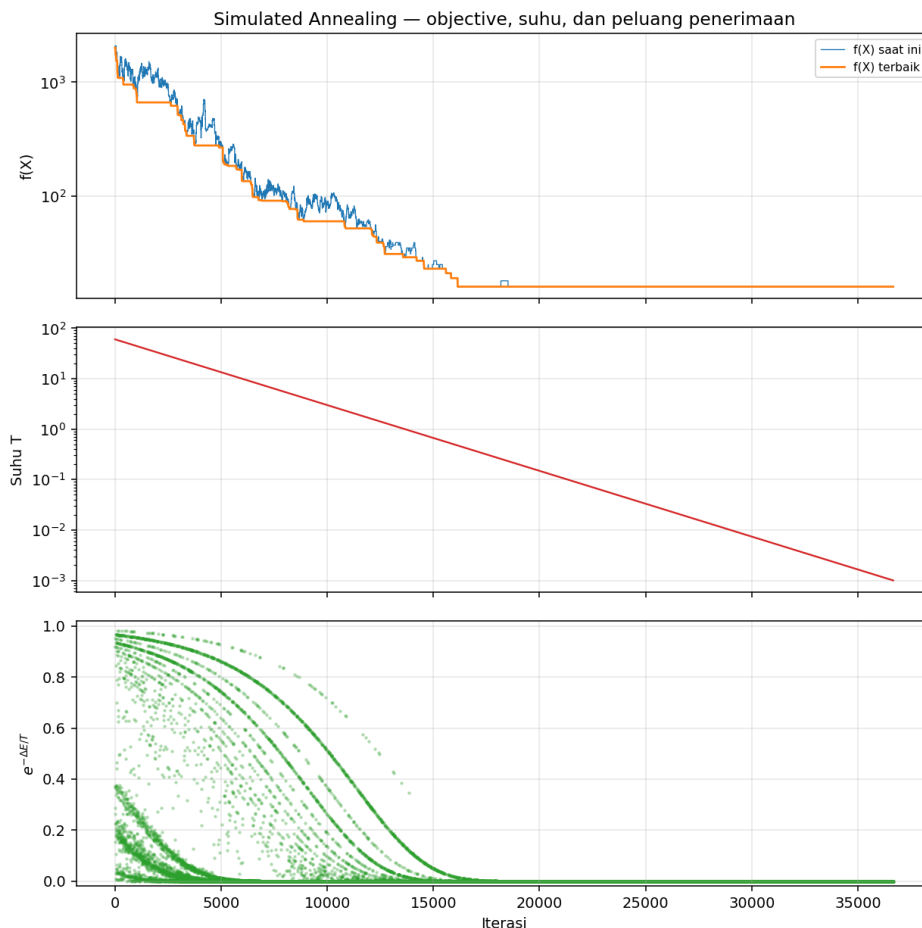
10.1 Eksperimen multi-seed

Satu kali jalan tidak cukup untuk menyimpulkan algoritma mana yang lebih baik. Tabel berikut merangkum sepuluh state awal berbeda (seed 1–10):

Algoritma	f rata-rata	Simpangan baku	f min	f maks	% feasible	Detik
Hill-Climbing (Sideways)	16,0	0,00	16	16	100%	7,68
Simulated Annealing	16,0	0,00	16	16	100%	1,74
Hill-Climbing (Random Restart)	16,2	0,63	16	18	100%	18,82
Genetic Algorithm	21,0	3,30	16	27	100%	1,55
Hill-Climbing (Steepest-Ascent)	45,7	41,57	16	134	60%	3,81
Hill-Climbing (Stochastic)	64,2	46,84	16	136	40%	1,38

Hasil ini mengubah kesimpulan yang akan diambil dari satu kali jalan saja. **Sideways Move dan Simulated Annealing sama-sama mencapai f = 16 pada seluruh sepuluh seed**, tetapi SA melakukannya empat kali lebih cepat dan dengan evaluasi f(X) sekitar empat kali lebih sedikit. Sebaliknya, Steepest-Ascent murni hanya menghasilkan jadwal feasible pada 6 dari 10 percobaan, ketergantungannya pada state awal sangat besar.

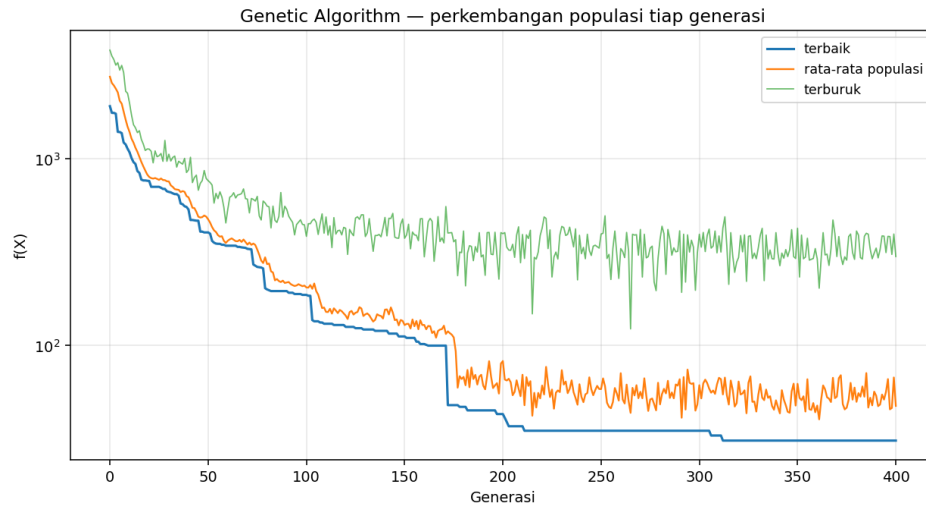
10.2 Diagnostik Simulated Annealing



Gambar 2. Tiga panel Simulated Annealing. Panel bawah menampilkan peluang penerimaan $e^{(-\Delta E/T)}$ untuk setiap langkah yang memperburuk objective: nilainya mendekati 1 di awal lalu meluruh ke 0 setelah iterasi ke-17.000, tepat ketika kurva $f(X)$ pada panel atas berhenti berfluktuasi dan mendatar.

Dari 32.071 langkah memburuk yang ditawarkan, 1.493 (4,7%) diterima, hampir seluruhnya pada paruh pertama pencarian. Sebanyak 21.062 penolakan terjadi saat suhu sudah di bawah 1, yaitu fase ketika SA praktis berperilaku seperti Hill-Climbing biasa. Uji sensitivitas dengan $T_0 \in \{20, 60, 150\}$ dan $\alpha \in \{0,999; 0,9995; 0,9997; 0,9999\}$ menghasilkan $f = 16$ pada hampir seluruh kombinasi, sehingga SA terbukti sangat tahan terhadap pemilihan hyperparameter pada persoalan ini.

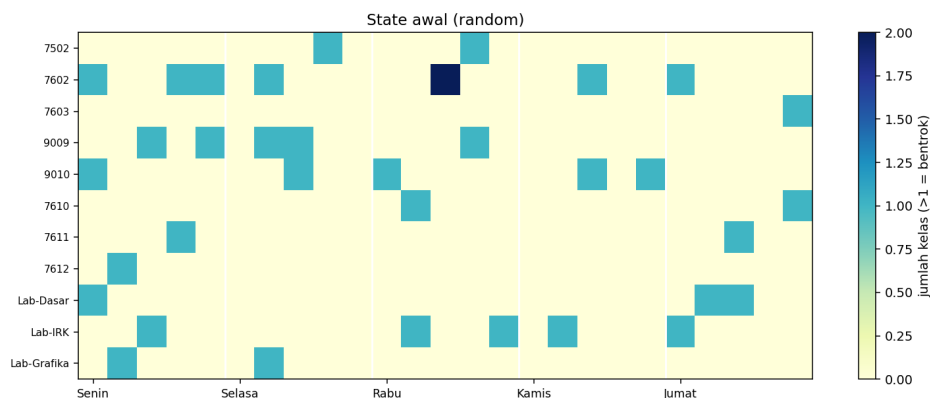
10.3 Diagnostik Genetic Algorithm



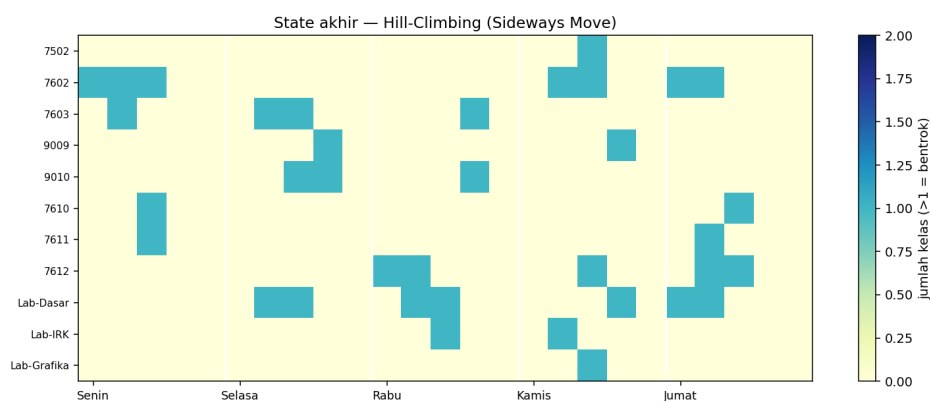
Gambar 3. Perkembangan populasi GA. Individu terbaik turun cepat pada 100 generasi pertama lalu nyaris mendatar, sementara rata-rata populasi tetap jauh di atasnya, tanda bahwa crossover terus menghasilkan keturunan yang jauh lebih buruk daripada elitennya.

GA paling sensitif terhadap hyperparameter di antara keenam algoritma. Uji petak menunjukkan f rata-rata 327 pada populasi 30 dengan mutasi 0,05, tetapi turun ke 16 pada populasi 160 dengan mutasi 0,5, rentang lebih dari dua puluh kali lipat. Penyebabnya adalah *uniform crossover*: karena kualitas jadwal bergantung pada kombinasi antar-gen (dua kelas boleh berada di ruang yang sama asalkan waktunya berbeda), menukar gen secara bebas kerap merusak kombinasi yang sudah baik. Populasi besar dan mutasi tinggi diperlukan justru untuk mengimbangi kerusakan itu.

10.4 Visualisasi perubahan state



Gambar 4. Okupansi ruangan pada state awal random. Warna gelap menandakan lebih dari satu kelas pada satu sel (ruang, slot), bentrok H1.



Gambar 5. Okupansi ruangan pada state akhir terbaik. Tidak ada sel bertumpuk, kolom terakhir tiap hari (sesi 15.00–17.00) kosong, dan pemakaian ruang terpusat pada ruangan yang kapasitasnya paling sesuai.

Proof of concept juga menghasilkan animasi GIF ([results/local_search/animasi_pencarian.gif](#)) yang menampilkan perubahan okupansi ruangan antar iterasi berdampingan dengan posisi iterasi tersebut pada kurva $f(X)$, dibuat lewat opsi `--animate`.

10.5 Ringkasan analisis

- **Steepest-Ascent** turun paling curam per iterasi (2015 → 18 hanya dalam 41 langkah) tetapi paling mahal: 1.854 evaluasi per iterasi. Ia berhenti tepat saat seluruh tetangga sama atau lebih buruk, definisi local optimum.
- **Sideways Move** membuktikan bahwa titik berhenti Steepest-Ascent memang sebuah **plateau**, bukan optimum sejati: dengan mengizinkan langkah mendatar, f turun lebih jauh dari 18 ke 16. Seluruh 40 kuota langkah mendatar terpakai.
- **Stochastic HC** paling murah per iterasi tetapi rasio penerimaannya hanya 0,3% (89 dari 30.000). Karena neighborhood berukuran 1.854 dan hanya sedikit yang membaik, mengundi satu tetangga acak nyaris selalu sia-sia setelah fase awal.
- **Random Restart** mencapai $f = 16$ dengan menjalankan lima pencarian independen (hasil per restart: 18, 16, 18, 77, 75). Ia paling andal secara probabilistik namun paling boros: 409.728 evaluasi, sebelas kali lipat SA.
- **Simulated Annealing** adalah pemenang keseluruhan pada persoalan ini: kualitas setara Sideways/Random Restart, 100% feasible di sepuluh seed, dengan hanya 36.670 evaluasi dan 1,7 detik.
- **Genetic Algorithm** paling cepat menurunkan f di awal karena mengevaluasi 80 individu sekaligus, tetapi mendatar paling dini. GA lebih cocok bila representasinya memiliki blok bangunan yang dapat diwariskan utuh, sesuatu yang tidak dimiliki representasi jadwal ini.

11. Struktur Repository dan Cara Menjalankan

```
Task2_AI_13524012/
|-- src/
|   |-- local_search/           # PoC Local Search
|   |   |-- problem.py         # state, constraints, objective, successor function
|   |   |-- algorithms.py      # 4 varian HC, Simulated Annealing, Genetic Algorithm
|   |   |-- visualize.py       # render jadwal, grafik, heatmap, animasi GIF
|   |   |-- main.py            # CLI
|   |-- dtl_lr_svm/            # implementasi DTL, LR, SVM
|-- notebooks/
|   |-- local_search/          # notebook eksperimen Local Search
|   |-- dtl_lr_svm/            # notebook eksperimen DTL, LR, SVM
|-- data/                      # train.csv, test.csv, sample_submission.csv
|-- results/                   # keluaran: grafik, jadwal, submission, JSON
|-- docs/
|   |-- Task2_AI_13524012.pdf  # dokumen ini
|-- README.md
```

Prasyarat: Python 3.10+ dengan numpy, pandas, dan matplotlib (pillow diperlukan untuk animasi GIF).
Seluruh perintah dijalankan dari direktori src/:

```
cd src
python -m local_search.main --algo all --seed 42 --plot      # seluruh algoritma + grafik
python -m local_search.main --algo sa --seed 7              # satu algoritma saja
python -m local_search.main --algo ga --pop 160 --mutation 0.5
```

12. Referensi

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021, ch. 4: Search in Complex Environments.
- [2] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: <https://www.youtube.com/watch?v=oGTa9kGE3P8>. [Accessed: Aug. 6, 2026].
- [3] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Aug. 6, 2026].
- [4] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: <https://www.youtube.com/watch?v=L0b0CN3Fds8>. [Accessed: Aug. 6, 2026].
- [5] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: <https://www.youtube.com/watch?v=AdhG7HnhYrk>. [Accessed: Aug. 6, 2026].
- [6] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: <https://www.youtube.com/watch?v=aI41TGo9J9c>. [Accessed: Aug. 6, 2026].
- [7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [8] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.
- [9] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, 2008.

Bagian II Write-up: DTL, Logistic Regression, dan SVM

Loan Approval Prediction, Kaggle Community Prediction Competition, Task #2 Seleksi Laboratorium Inteligensi Buatan. Tengku Naufal Saqib (13524012).

1. TLDR

Tiga algoritma (**Decision Tree Learning (CART)**, **Logistic Regression**, dan **Support Vector Machine**) diimplementasikan dari nol dengan numpy saja, lalu dibandingkan dengan padanannya di scikit-learn memakai 5-fold stratified cross-validation atas 28.800 baris data latih. **CART unggul telak** dengan akurasi **0,9160**, sementara Logistic Regression dan SVM linier mentok di kisaran 0,897–0,900. Ketiga implementasi *from scratch* setara dengan scikit-learn (selisih $< 0,001$; untuk CART bahkan identik sampai empat desimal), sehingga perbedaan performa antar algoritma murni berasal dari sifat algoritmanya, bukan dari kualitas implementasi. Penyebab keunggulan CART jelas: dataset ini didominasi satu aturan deterministik dan sejumlah **interaksi** antar atribut yang tidak dapat ditangkap model linier. Dugaan itu diuji secara langsung: menambahkan aproksimasi kernel RBF lewat *Random Fourier Features* menaikkan Logistic Regression dari 0,8973 ke 0,9047, dan SVM dual berkernel RBF mencapai 0,914 hanya dengan 1.875 sampel latih. Model yang disubmit ke Kaggle adalah CART tunggal dengan *cost-complexity pruning*.

2. Problem Overview

Persoalannya klasifikasi biner: memprediksi apakah sebuah pengajuan pinjaman disetujui ($\text{loan_status} = 1$). Data latih berisi **28.800** baris dan data uji **7.200** baris, dengan 11 atribut prediktor: 8 numerik (umur, pendapatan, lama bekerja, besar pinjaman, suku bunga, rasio pinjaman terhadap pendapatan, panjang riwayat kredit, skor kredit) dan 3 kategorik (jenis kelamin, status kepemilikan rumah, riwayat gagal bayar). Tidak ada nilai kosong sama sekali. Kelas positif hanya **22,2%**, sehingga akurasi harus dibaca berdampingan dengan F1.

Tiga temuan EDA mengarahkan seluruh pengerjaan. **Pertama**, terdapat anomali nilai: umur maksimum 144 tahun, lama bekerja maksimum 125 tahun, dan pendapatan maksimum 7,2 juta padahal mediana 67 ribu; semuanya dipangkas ke batas wajar, dan lama bekerja dibatasi agar tidak melebihi $\text{umur} - 16$. **Kedua**, dan yang paling penting, **seluruh 14.629 baris dengan riwayat gagal bayar bernilai "Yes" berlabel 0 tanpa kecuai**, separuh dataset praktis ditentukan oleh satu aturan deterministik. **Ketiga**, prediktor numerik terkuat adalah rasio pinjaman terhadap pendapatan (korelasi 0,385) dan suku bunga (0,335); sisanya lemah, dengan skor kredit hampir tidak berkorelasi sama sekali ($-0,005$).

Dua belas fitur turunan ditambahkan, terutama rasio yang lazim dipakai analisis risiko (log pendapatan, beban bunga terhadap pendapatan, sisa pendapatan setelah pinjaman, usia mulai bekerja, rasio riwayat kredit terhadap umur). Setelah encoding biner dan one-hot status kepemilikan rumah, tersisa **26 fitur**. Standarisasi z-score dipelajari **hanya** dari lipatan latih, sehingga tidak ada kebocoran informasi.

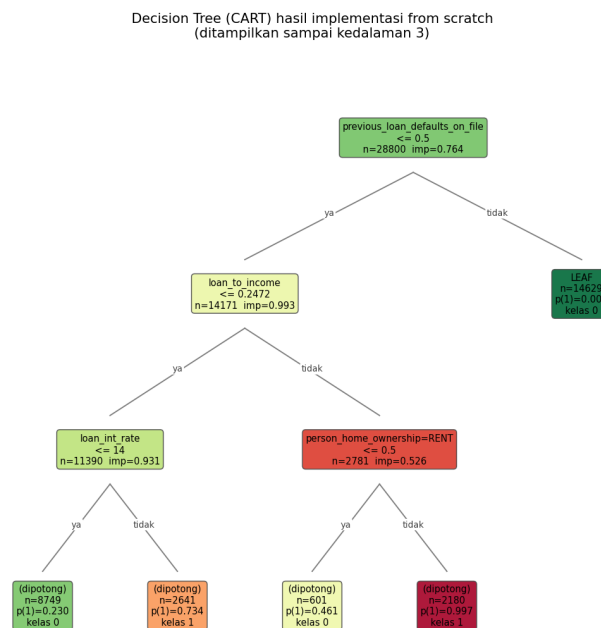
3. Decision Tree Learning (CART)

Pemilihan varian. Dari ID3, C4.5, dan CART, dipilih **CART** karena sebelas dari tiga belas atribut bersifat kontinu. ID3 sama sekali tidak menangani atribut kontinu sehingga memerlukan diskretisasi manual yang membuang informasi urutan. C4.5 memang mendukung atribut kontinu, tetapi keunggulan utamanya (*gain ratio* untuk mengoreksi bias terhadap atribut ber-kardinalitas tinggi) tidak relevan di sini karena setelah encoding hanya tersisa atribut numerik dan biner. Sebaliknya, *binary split* milik CART mencari ambang optimum langsung pada nilai kontinu, dan struktur binernya membuat *cost-complexity pruning* mudah diterapkan.

Implementasi. Pencarian split memakai jumlah kumulatif atas nilai fitur yang sudah diurutkan sehingga seluruh kandidat ambang dievaluasi serentak; kompleksitasnya $O(d \cdot n \log n)$ per simpul, bukan $O(d \cdot n^2)$, dan satu pohon selesai dalam 0,3 detik. Gini dan entropi keduanya tersedia; bobot kelas ditangani lewat bobot per sampel sehingga tidak perlu menduplikasi baris.

Hasil penelusuran. Akurasi naik sampai $\text{max_depth} = 8$ lalu **menurun** (0,9157 pada kedalaman 8, 0,9113 pada kedalaman 12), tanda overfitting yang jelas. Entropi unggul tipis dan konsisten atas Gini di hampir seluruh kedalaman. **Cost-complexity pruning** (Breiman dkk., 1984) memberi bukti overfitting yang paling gamblang: pohon berkedalaman 12 dengan 229 daun dipangkas menjadi 20 daun oleh $\alpha = 10^{-3}$, dan akurasinya justru **naik** dari 0,9113 ke 0,9153. Konfigurasi akhir (kedalaman 8, $\text{min_samples_leaf} = 5$, entropi, $\alpha = 5 \times 10^{-4}$, 32 daun) mencapai **0,9160**.

Struktur pohonnya sangat mudah dibaca. Split akar adalah riwayat gagal bayar, dan cabang "Yes" langsung menjadi **daun murni** berisi 14.629 sampel dengan $p(1) = 0,000$: algoritma menemukan sendiri aturan deterministik yang terlihat pada EDA. Sisa pohon bekerja pada separuh data yang benar-benar ambigu, dipimpin rasio pinjaman terhadap pendapatan, suku bunga, dan status menyewa rumah.



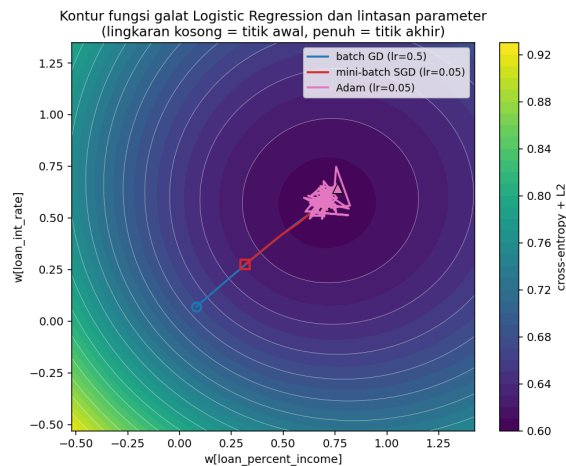
Gambar 6. Percabangan CART hasil implementasi from scratch (kedalaman 0–3). Warna menyatakan proporsi kelas positif pada simpul.

4. Logistic Regression

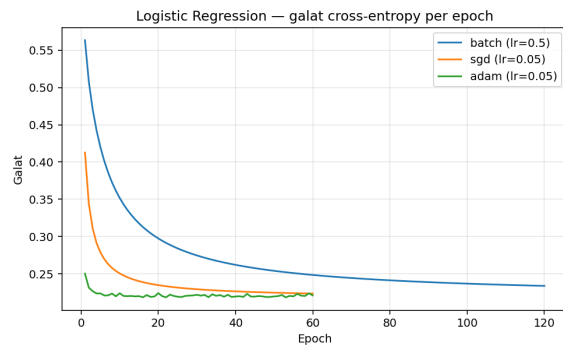
Model memaksimalkan log-likelihood Bernoulli, ekuivalen dengan meminimalkan *binary cross-entropy* berbobot kelas ditambah regularisasi L2. Gradiennya berbentuk sederhana karena turunan sigmoid dan cross-entropy saling meniadakan: $\partial J / \partial w = X^T (c \odot (p - y)) / n + \lambda w$. Sigmoid diimplementasikan dalam bentuk yang stabil secara numerik (cabang terpisah untuk $z \geq 0$ dan $z < 0$) agar tidak *overflow*.

Tiga *optimizer* dibandingkan. **Batch gradient descent** memerlukan 150 epoch untuk mendekati konvergensi. **Mini-batch SGD** mencapai galat yang sama dalam 60 epoch. **Adam** (Kingma & Ba, 2015), algoritma optimasi tambahan di luar materi kelas, memelihara rata-rata bergerak momen pertama m (arah gradien) dan momen kedua v (skala kuadrat gradien), keduanya dikoreksi bias, lalu melangkah sebesar $\alpha * m_hat / (\sqrt{v_hat} + \epsilon)$. Karena tiap parameter memperoleh laju belajar efektifnya sendiri, Adam tidak terhambat fitur dengan skala gradien kecil dan menghasilkan akurasi CV tertinggi di antara ketiganya (0,8973 melawan 0,8968).

Untuk melihat proses pelatihannya secara langsung, model dilatih ulang hanya dengan dua fitur terkuat dan **tanpa intercept**, sehingga ruang parameternya memang berdimensi dua dan kontur yang digambar benar-benar fungsi galat yang dioptimasi, bukan proyeksi atau aproksimasi.



Gambar 7. Kontur cross-entropy + L2 dan lintasan tiga optimizer (lingkaran kosong = titik awal, penuh = titik akhir). Batch GD bergerak lurus namun lambat dan setelah 150 epoch belum sampai ke minimum; Adam tiba di cekungan minimum paling cepat lalu berosilasi kecil.



Gambar 8. Galat per epoch. Adam dan mini-batch SGD mencapai galat yang sama dengan batch GD dalam jumlah epoch jauh lebih sedikit.

Regularisasi L2 diuji pada rentang 0 sampai 10^{-1} ; nilai terbaik 10^{-4} dengan pengaruh kecil ($< 0,002$), wajar karena 26 fitur atas 28.800 sampel tidak menghadirkan risiko overfitting yang berarti pada model linier.

5. Support Vector Machine

SVM diimplementasikan dalam dua formulasi. **Primal:** label diubah ke $\{-1, +1\}$ dan fungsi objektif *soft-margin* $(\lambda/2)\|w\|^2 + \text{mean}(\text{hinge})$ diminimalkan lewat sub-gradien; hanya sampel yang melanggar margin ($t \cdot f(x) < 1$) yang berkontribusi. **Dual:** *simplified SMO* (Platt, 1998), yang mengoptimasi dua pengali Lagrange secara analitik pada tiap langkah sambil menjaga kendala $\sum \alpha_i t_i = 0$ dan $0 \leq \alpha \leq C$.

Sebagai algoritma optimasi tambahan dipakai **Pegasos** (Shalev-Shwartz dkk., 2011), yang memakai laju belajar meluruh sendiri $\eta_t = 1/(\lambda t)$ lalu memproyeksikan w ke bola berjari-jari $1/\sqrt{\lambda}$, tempat solusi optimum dijamin berada. Jumlah iterasi yang dibutuhkan untuk mencapai galat ϵ adalah $O(1/(\lambda \epsilon))$, **tidak bergantung pada jumlah data**. Hasilnya menarik dan sempat mengecoh: pada $\lambda = 10^{-4}$ Pegasos justru **kalah** dari sub-gradien biasa (0,874 melawan 0,895), karena langkah pertamanya $\eta_1 = 10^4$ terlalu besar dan radius proyeksi 100 terlalu longgar untuk menahannya. Menaikkan λ ke 10^{-3} menjadikan Pegasos yang terbaik (**0,8998**, sedikit di atas LinearSVC). Pelajarannya: klaim "bebas hyperparameter" pada Pegasos hanya berlaku untuk laju belajar, bukan untuk λ .

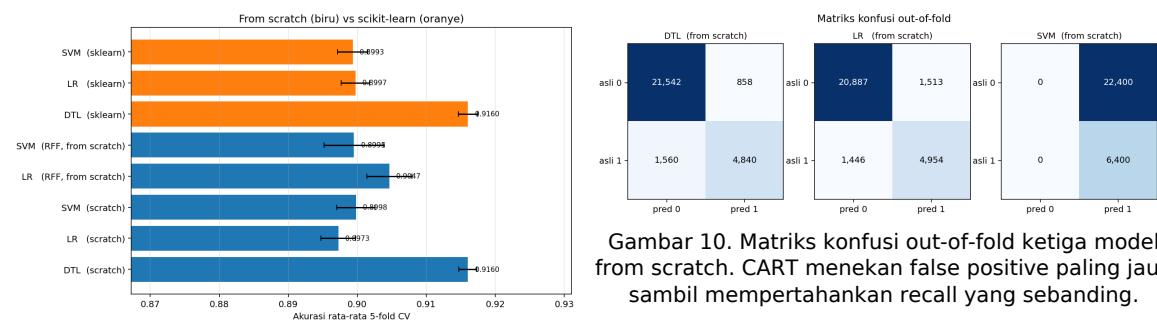
SVM kernel penuh tidak mungkin dijalankan di sini karena matriks Gram 28.800×28.800 memerlukan $\sim 6,6$ GB. Karena itu dipakai **Random Fourier Features** (Rahimi & Recht, 2007): kernel Gaussian didekati sebagai hasil kali dalam pada ruang eksplisit berdimensi D lewat $z(x) = \sqrt{(2/D)} \cdot \cos(Wx + b)$, $W \sim N(0, 2\gamma)$, $b \sim U(0, 2\pi)$, sehingga biayanya turun dari $O(n^2)$ ke $O(nD)$. Dengan $D = 800$ dan $\gamma = 0,03$, Logistic Regression naik ke **0,9047**. SMO berkernel RBF pada subsampel 1.875 baris mencapai **0,9136** (600 *support vector*) melawan hanya 0,8560 untuk kernel linier pada data yang sama. Keduanya menunjuk kesimpulan yang sama: batas keputusannya memang non-linier.

6. Validation Strategy

Seluruh keputusan diambil berdasarkan **5-fold stratified cross-validation** yang juga ditulis dari nol: indeks tiap kelas diacak lalu dibagi rata ke lima lipatan sehingga proporsi 22,2% kelas positif terjaga di setiap lipatan. Stratifikasi penting karena kelas tidak seimbang, pembagian acak biasa dapat menghasilkan lipatan dengan proporsi positif menyimpang beberapa persen dan membuat perbandingan antar model tidak stabil. Seed 42 dipakai konsisten sehingga seluruh model dievaluasi pada partisi yang **persis sama**. Metrik yang dilaporkan: akurasi, presisi, recall, dan F1, seluruhnya dihitung oleh implementasi sendiri di `metrics.py`.

Model	Akurasi CV	F1	Presisi	Recall	Skor Kaggle
DTL (from scratch)	0,9160 ± 0,0013	0,8001	0,8496	0,7562	0,86013
DTL (sklearn)	0,9160 ± 0,0014	0,7990	0,8536	0,7511	-
LR + RFF (from scratch)	0,9047 ± 0,0032	0,7715	0,8260	0,7250	-
SVM (from scratch, Pegasos)	0,8998 ± 0,0028	0,7727	0,7788	0,7670	-
LR (sklearn)	0,8997 ± 0,0021	0,7736	0,7762	0,7711	-
SVM + RFF (from scratch)	0,8995 ± 0,0044	0,7512	0,8349	0,6830	-
SVM (sklearn, LinearSVC)	0,8993 ± 0,0022	0,7735	0,7736	0,7734	-
LR (from scratch, Adam)	0,8973 ± 0,0025	0,7700	0,7664	0,7741	-

Simpangan baku antar lipatan hanya 0,0013–0,0044, jadi selisih 0,016 antara CART dan SVM jauh melampaui derau, kesimpulan bahwa pohon menang bersifat kokoh. Skor leaderboard untuk CART adalah **0,86013**, di bawah akurasi validasi lokalnya 0,9160. Selisih 5,6 poin itu lebih besar daripada ragam antar lipatan, sehingga bukan sekadar derau pembagian data; definisi metrik resmi kompetisi dan format berkas keluaran perlu diperiksa lebih lanjut sebelum penyetulan berikutnya dilakukan.



Gambar 9. Akurasi 5-fold CV: implementasi from scratch (biru) melawan scikit-learn (oranye). Selisihnya di bawah 0,001 untuk ketiga algoritma.

Gambar 10. Matriks konfusi out-of-fold ketiga model from scratch. CART menekan false positive paling jauh sambil mempertahankan recall yang sebanding.

7. Percobaan yang Gagal

- **Pegasos dengan λ kecil.** Percobaan pertama memakai $\lambda = 10^{-4}$ mengikuti regularisasi Logistic Regression, dan hasilnya 0,874, di bawah sub-gradien biasa. Baru setelah menyadari bahwa pada Pegasos λ ikut menentukan besar langkah *dan* radius proyeksi, sapuan nilai λ dilakukan dan 10^{-3} terpilih (0,8998).
- **Cost-complexity pruning yang tidak berefek.** Implementasi pertama memakai risiko tak ternormalisasi $R(t) = n_t \cdot i(t)$, sehingga seluruh nilai α yang wajar (10^{-4} – 10^{-2}) tidak memangkas satu simpul pun; pohon tetap 229 daun untuk setiap α . Setelah risiko dinormalisasi terhadap jumlah sampel, $R(t) = (n_t/N) \cdot i(t)$, mengikuti konvensi scikit-learn, pemangkasan berjalan dan justru menaikkan akurasi.
- **Random Fourier Features untuk SVM.** RFF menaikkan Logistic Regression sebesar 0,007 tetapi **tidak menolong SVM sama sekali** (0,8998 $\rightarrow$ 0,8995) sambil menurunkan F1 dari 0,773 ke 0,751 dan menaikkan waktu latih dua puluh kali lipat: hinge loss hanya memperoleh sinyal dari pelanggar margin, sehingga 800 dimensi acak justru mengencerkannya. Menaikkan γ ke 0,1 memperburuk keduanya (0,873 SVM / 0,879 LR) karena kernel menjadi terlalu sempit.
- **Rekayasa fitur untuk pohon.** Dua belas fitur turunan menaikkan Logistic Regression (0,8909 $\rightarrow$ 0,8973) dan SVM (0,8927 $\rightarrow$ 0,8998), tetapi untuk CART hasilnya **0,9162 tanpa fitur turunan melawan 0,9160 dengan fitur turunan** selisihnya di dalam derau, dan arahnya justru sedikit merugikan. Masuk akal: rasio-rasio itu dibuat agar model linier dapat menyentuh interaksi, sedangkan pohon sudah menemukan interaksi yang sama lewat pemecahan berurutan. Pipeline tunggal tetap dipertahankan agar seluruh model dibandingkan atas representasi yang sama.
- **Penyetelan ambang keputusan.** Mencari ambang optimum pada skor out-of-fold hanya menaikkan akurasi dari 0,9157 ke 0,9161, kurang dari satu simpangan baku antar lipatan. Karena keuntungannya sebanding dengan risiko menyetel ambang pada data yang sama, ambang 0,5 dipertahankan.

8. Pengembangan Lebih Lanjut

- **Kernel SVM penuh dengan SMO ter-cache.** SMO berkernel RBF mencapai 0,9136 hanya dengan 1.875 sampel latih. Versi lengkapnya (*error cache* + heuristik pemilihan pasangan milik Platt + dekomposisi subset aktif) memungkinkan pelatihan pada seluruh 28.800 sampel dan berpeluang melampaui CART.
- **Post-pruning berbasis validasi.** Saat ini α dipilih lewat sapuan manual. Membangkitkan seluruh jalur pemangkasan *weakest-link* secara otomatis (Breiman dkk., Bab 3) lalu memilih α dengan aturan satu simpangan baku akan menghasilkan pohon yang lebih ringkas dengan jaminan generalisasi lebih baik.
- **Interaksi eksplisit untuk model linier.** Ekspansi polinomial derajat dua atas enam fitur terkuat merupakan jalan tengah antara model linier dan RFF: ia menambahkan interaksi yang *dapat ditafsirkan*, bukan 800 dimensi acak.
- **Verifikasi metrik resmi kompetisi.** Selisih antara akurasi validasi lokal dan skor leaderboard belum terjelaskan. Memastikan definisi metrik serta format berkas keluaran yang diharapkan adalah langkah pertama sebelum penyetelan lanjutan, karena menyetel model pada metrik yang keliru membuang seluruh usaha berikutnya.
- **Penanganan ketidakseimbangan kelas.** `class_weight="balanced"` sudah tersedia pada ketiga implementasi namun belum ditelusuri sistematis; ia berpotensi menaikkan F1 meski menurunkan akurasi.

9. Referensi

- [1] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984. (CART, impuritas Gini, dan cost-complexity pruning pada Bab 2 dan 3)
- [2] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, San Diego, CA, USA, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>
- [3] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011. doi: 10.1007/s10107-010-0420-4
- [4] A. Rahimi and B. Recht, "Random Features for Large-Scale Kernel Machines," in *Advances in Neural Information Processing Systems 20 (NeurIPS)*, 2007, pp. 1177–1184.
- [5] J. C. Platt, "Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines," Microsoft Research, Tech. Rep. MSR-TR-98-14, 1998.
- [6] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009, ch. 9 (CART) and ch. 12 (SVM).
- [8] GeeksforGeeks, "CART (Classification And Regression Tree) in Machine Learning"; "Understanding Logistic Regression"; "Support Vector Machine (SVM) Algorithm." [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/>. [Accessed: Aug. 6, 2026].